Numpy and Pandas

- Array objects are multidimensional tensors with with a single data type.

- Pandas dataframes represent tables and allows seperate data types for each column.

- Both arrays and dataframes allow broadcasting.

Numpy arrays

```
import numpy as np

a = np.array([5, 12, 7])
```

Arrays

```
a = np.array([5, 12, 7])

a.dtype # → dtype('int64')
```

```
a = np.array([2.1, 9.3, 5.8])

a.dtype # → dtype(‘float64')

a = np.array([2.7, {'Bo': 42}]) # don’t!
# → dtype('O')
```

Indexing and slicing

```
a = np.array([5, 12, 7, 4, 6, 1])

a[1] # → np.int64(12)
a[1:4] # → np.array([12, 7, 4])
```

```
i = [0, 5, 4]
a[i] # → np.array([5, 1, 6])

b = a < 7
a[b] # → np.array([5, 4, 6, 1])
```

Broadcasting

```
a = np.array([1, 2])
b = np.array([3, 4])

a + b  # → np.array([4, 6])
a * b  # → np.array([3, 8])
a + 10 # → np.array([11, 12])
a > 1  # → np.array([False, True])


np.log(a) # → array([0., 0.69314718])
```

Multi-dimensional arrays

```
lists = [[1, 2, 3],
         [4, 5, 6],
         [7, 8, 9]]
```

```
np.array(lists)
# → np.array([[1, 2, 3],
#             [4, 5, 6],
#             [7, 8, 9]])
```

Numpy

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

matrix[0]         # → np.array([4, 5, 6])
matrix[0, :]      # → np.array([4, 5, 6])
matrix[:, 1]    # → np.array([2, 5, 8])
matrix[1, 1]      # → 5
matrix[:2, 1:] # → np.array([[2, 3],
#                             [5, 6]])
```

Methods

```
matrix.sum()
np.array([[1, 2, 3],
          [1, 2, 3],
          [1, 2, 3]]).sum()
# returns: 18
```

Axis argument

```
np.array([[1, 2, 3],
          [1, 2, 3],
          [1, 2, 3]]).sum(axis=0)
# returns: np.array([6, 6, 6])
```

```
np.array([[1, 2, 3],
          [1, 2, 3],
          [1, 2, 3]]).sum(axis=1)
# returns: np.array([3, 6, 9])
```

Transpose

```
np.array([[1, 2, 3],
          [1, 2, 3],
          [1, 2, 3]]).T

# → np.array([[1, 1, 1],
#             [2, 2, 2],
#             [3, 3, 3]])
```

Concatenate

```
a = np.array([1, 2])
b = np.array([3, 4])

np.concatenate([a, b])
# → np.array([1, 2, 3, 4])
```

Concatenate

```
m1 = np.array([[1, 2], [3, 4]])
m2 = np.array([[5, 6], [7, 8]])

np.concatenate([m1, m2], axis=0)
# → array([[1, 2],
#          [3, 4],
#          [5, 6],
#          [7, 8]])

np.concatenate([m1, m2], axis=1)
# → array([[1, 2, 5, 6],
#          [3, 4, 7, 8]])
```

Reshape

```
# row vector:
a = np.array([7, 2, 5])

# column vector:
c = a[np.newaxis, :]
# → np.array([[7],
#             [2]
#             [5]])

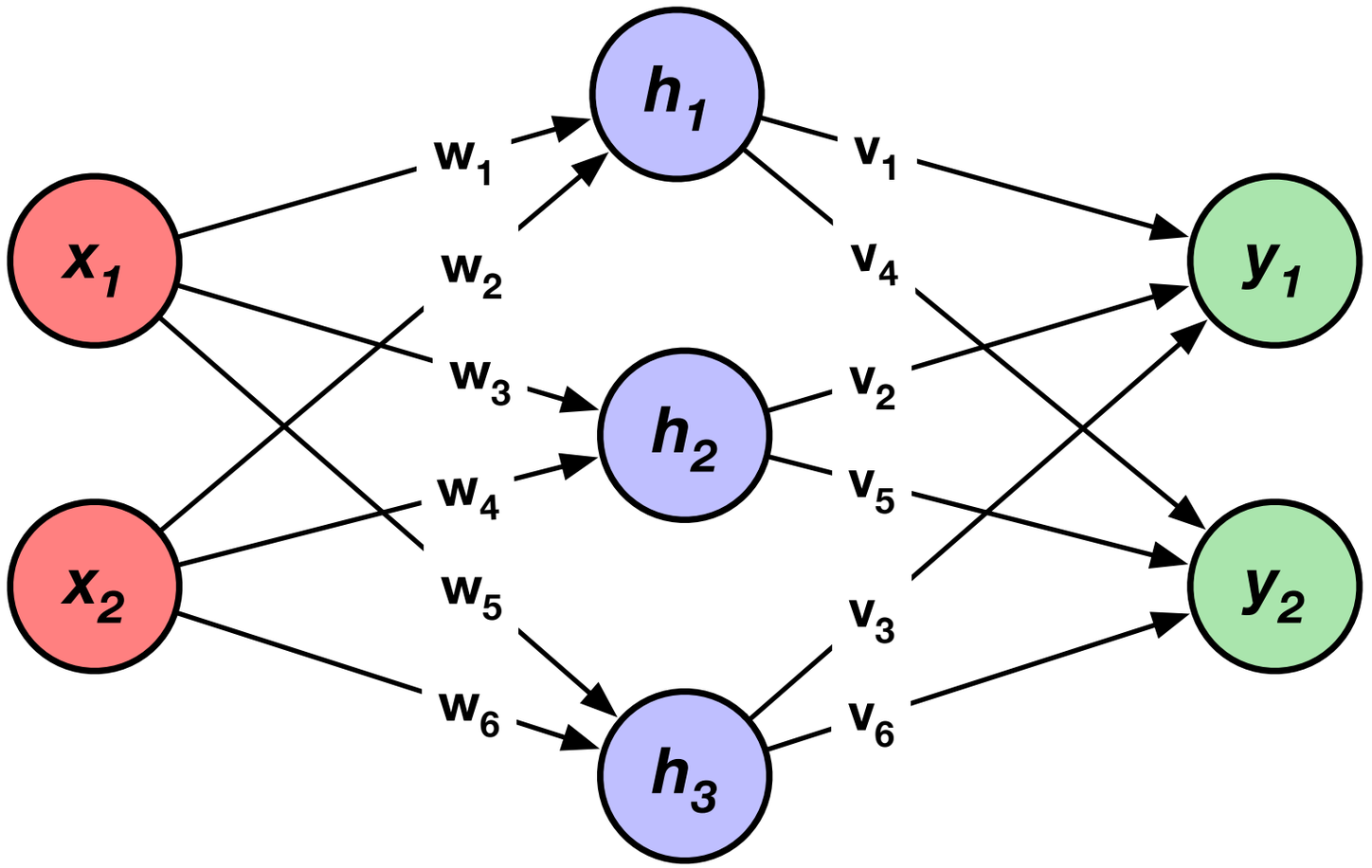
np.squeeze(c)
# → np.array([7, 2, 5])
```

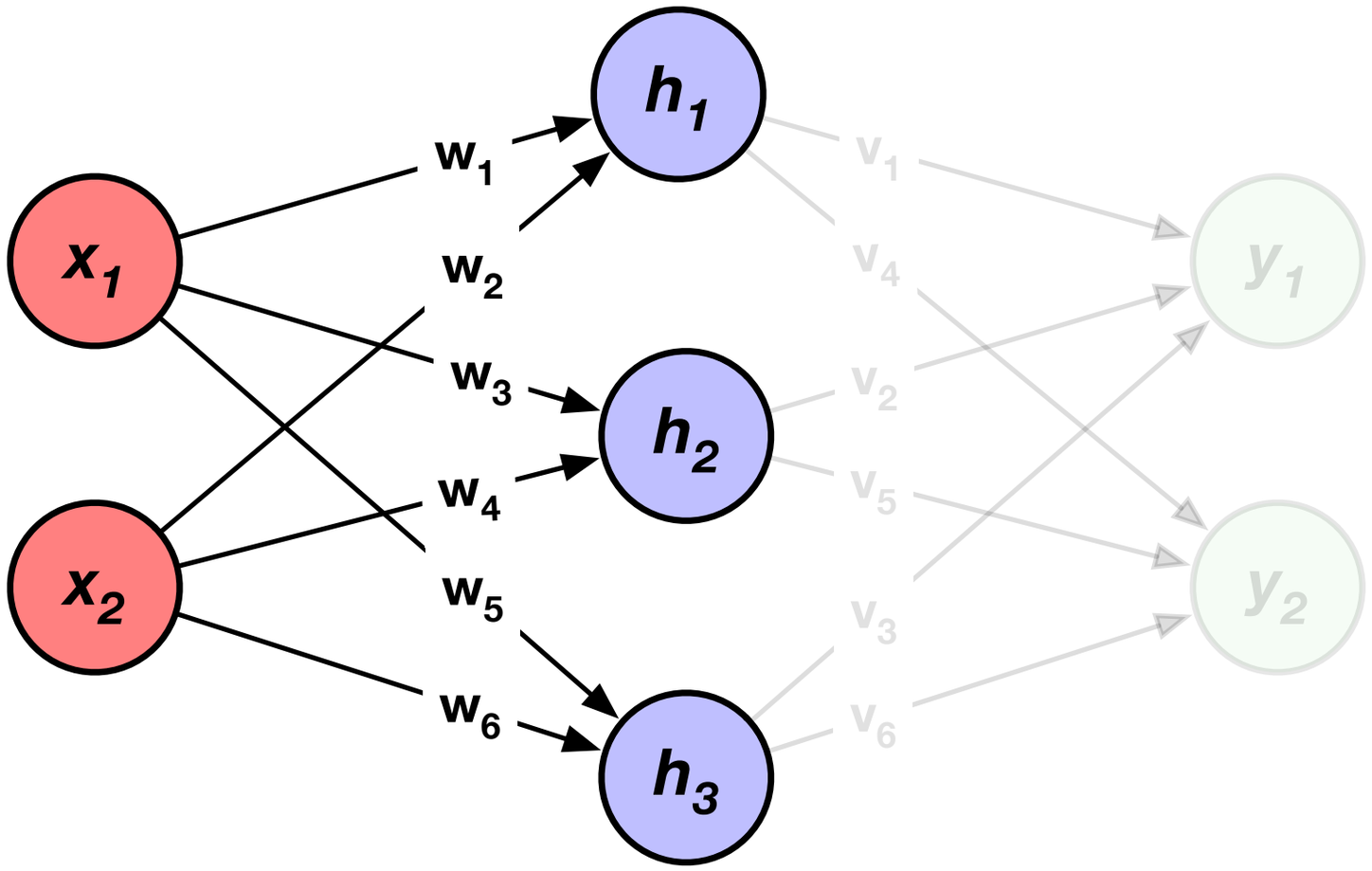
Hadamard vs. matrix product

```
m = np.array([[1, 2],
              [3, 4]])

m * m                                  # Hadamard product
# → np.array([[1,  4],
              [9, 16]])

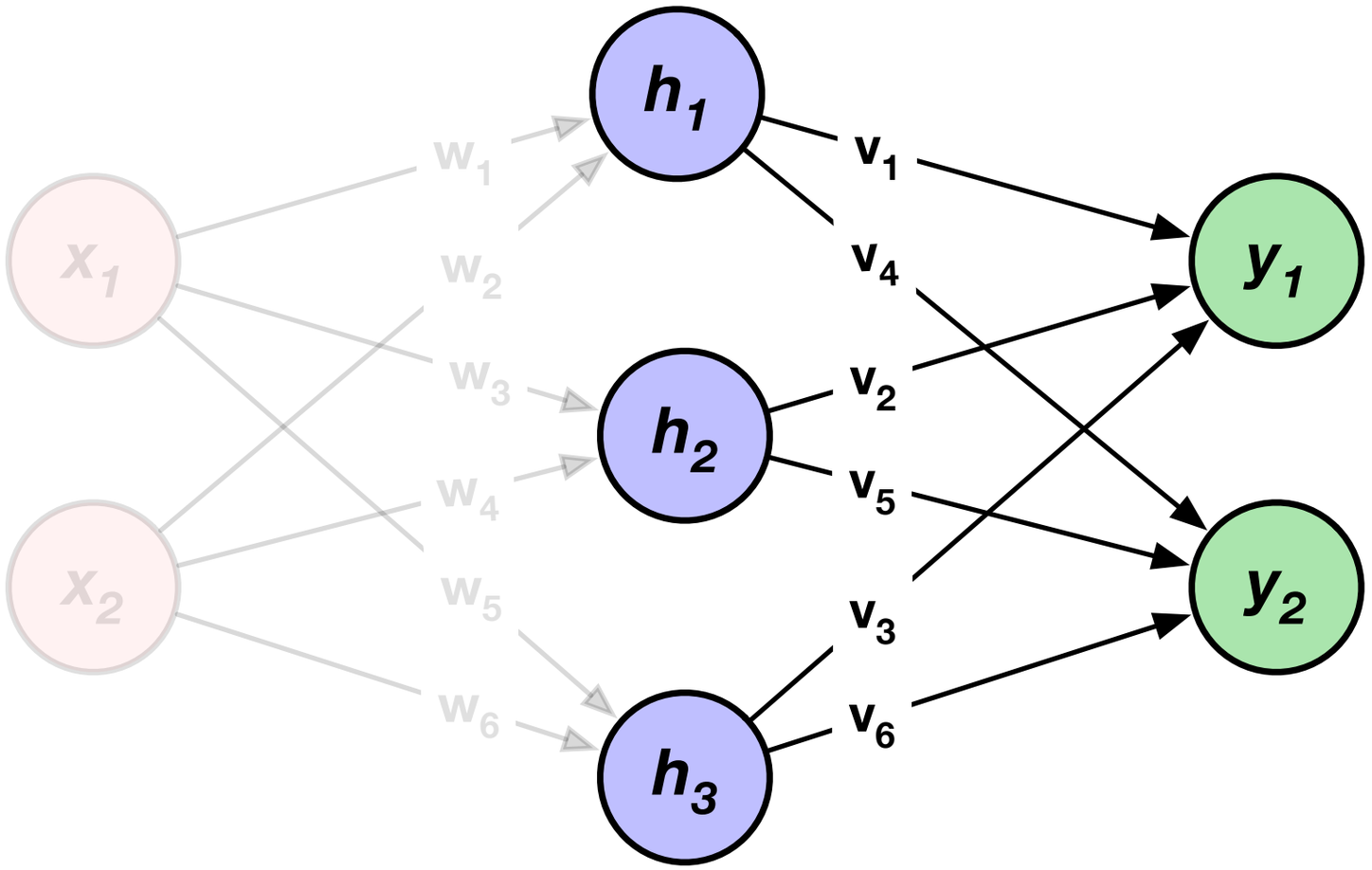

np.matmul(m, m)                        # Matrix product
# → np.array([[ 7, 10],
              [15, 22]])
```


h_1
w_1
x_1
w_2
w_3
h_2
w_4
x_2
w_5
w_6
h_3
v_1
v_4
y_1
v_2
v_5
y_2
v_3
v_6



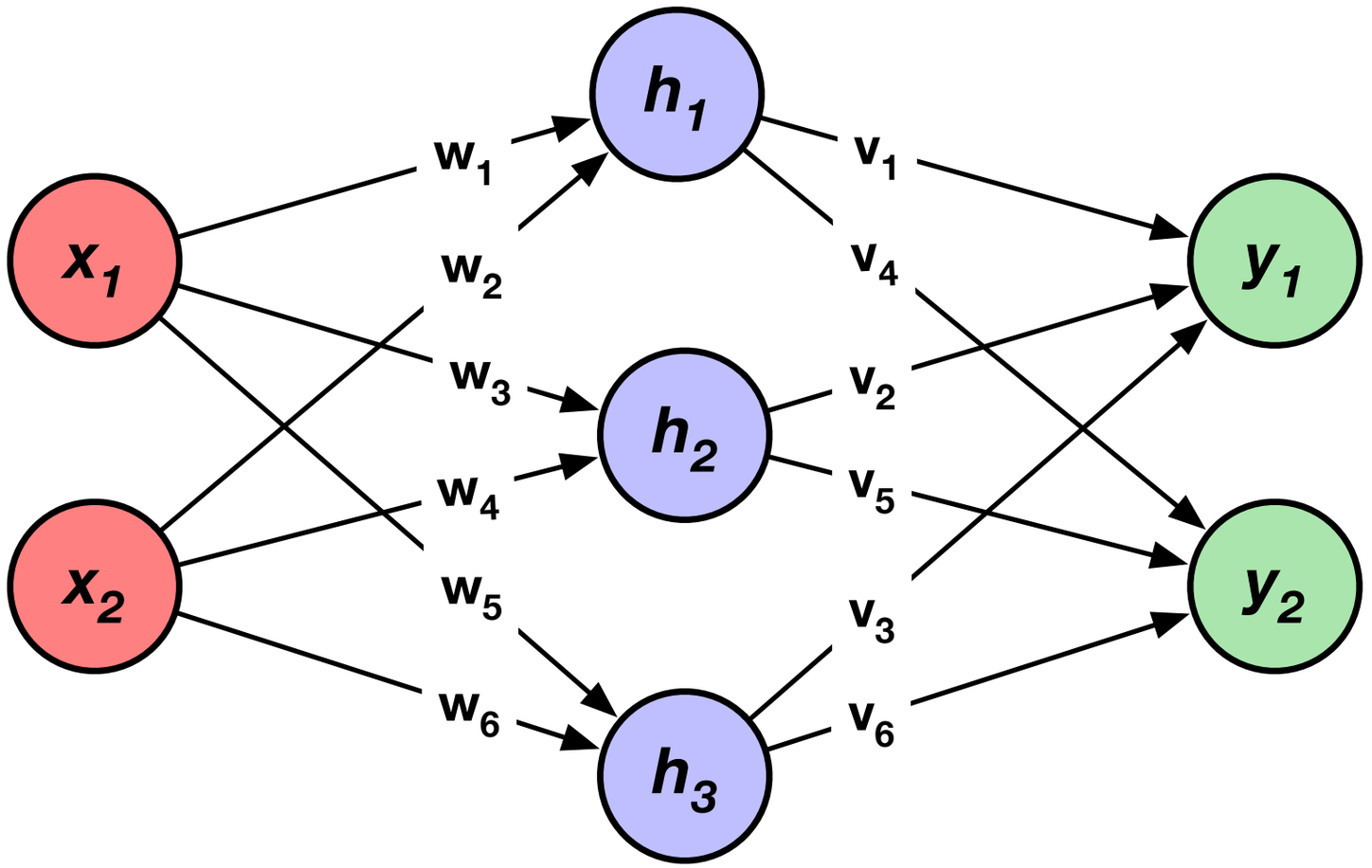
$$f\left(\begin{array}{c}\boldsymbol{X}\\ \begin{array}{|c|c|}\hline x_1 & x_2 \\ \hline\end{array}\end{array} \times \begin{array}{c}\boldsymbol{W}\\ \begin{array}{|c|c|c|}\hline w_1 & w_3 & w_5 \\ \hline w_2 & w_4 & w_6 \\ \hline\end{array}\end{array} - \begin{array}{c}\boldsymbol{\theta}\\ \begin{array}{|c|c|c|}\hline \theta_1 & \theta_2 & \theta_3 \\ \hline\end{array}\end{array}\right) = \begin{array}{c}\boldsymbol{H}\\ \begin{array}{|c|c|c|}\hline h_1 & h_2 & h_3 \\ \hline\end{array}\end{array}$$

$$f(XW - \theta) = H$$

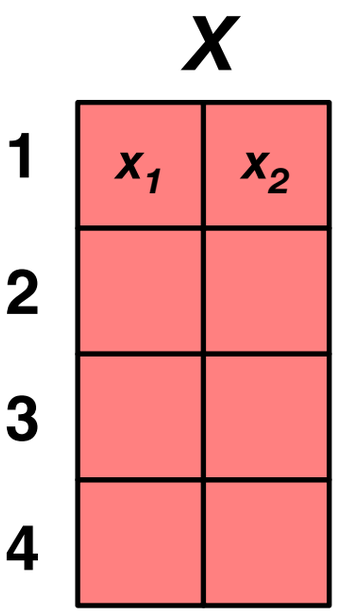


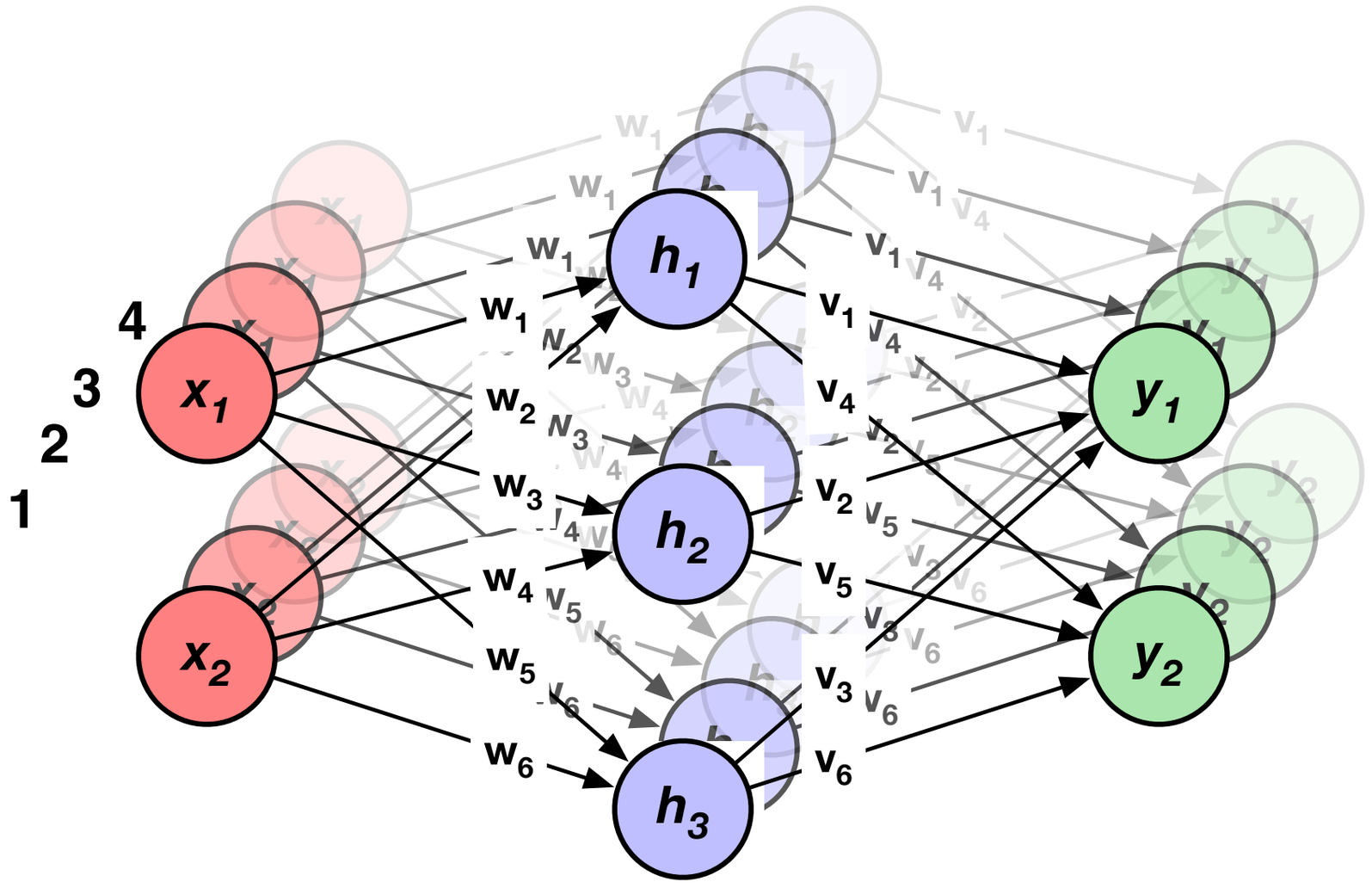
$$f\left(\begin{array}{|c|c|c|} \multicolumn{3}{c}{H} \\ \hline h_1 & h_2 & h_3 \end{array} \times \begin{array}{|c|c|} \multicolumn{2}{c}{V} \\ \hline v_1 & v_4 \\ \hline v_2 & v_5 \\ \hline v_3 & v_6 \end{array} - \begin{array}{|c|c|} \multicolumn{2}{c}{\theta} \\ \hline \theta_1 & \theta_2 \end{array} \right) = \begin{array}{|c|c|} \multicolumn{2}{c}{Y} \\ \hline y_1 & y_2 \end{array}$$

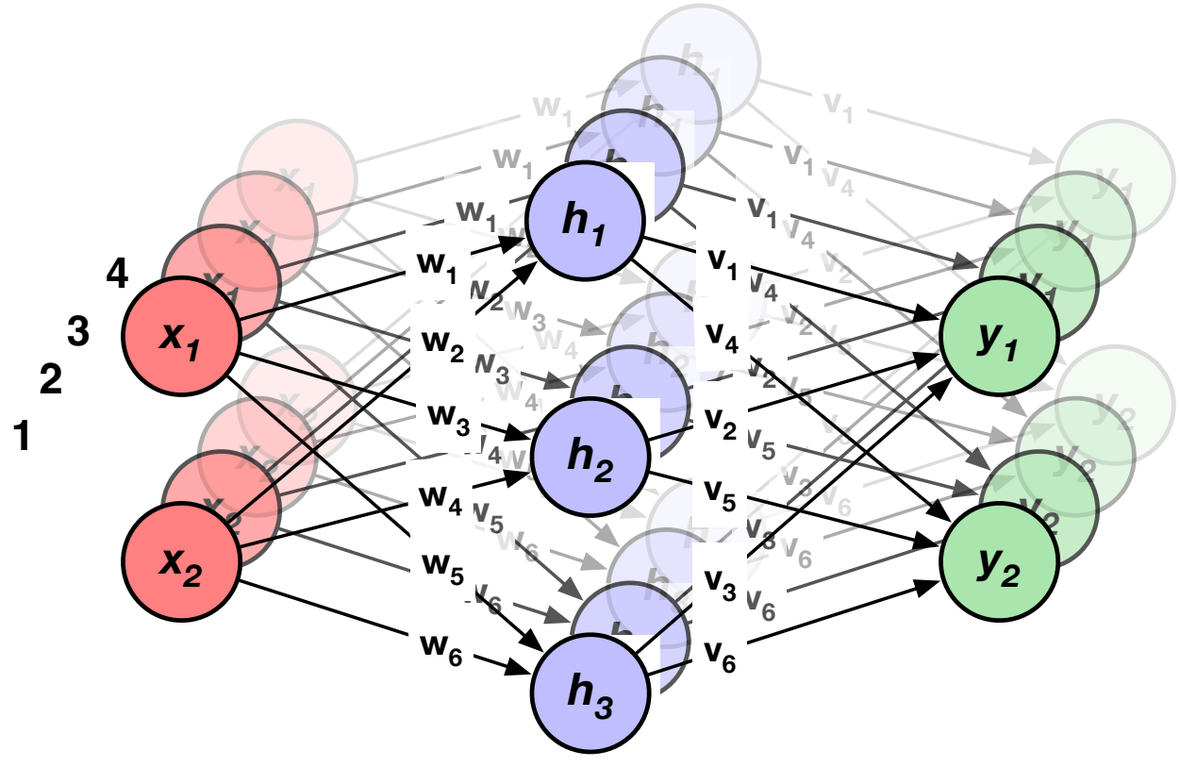
$$f(HV - \theta) = Y$$



$$f(XW - \theta) = H \qquad f(HV - \theta) = Y$$


X
1
2
3
4
x_1
x_2

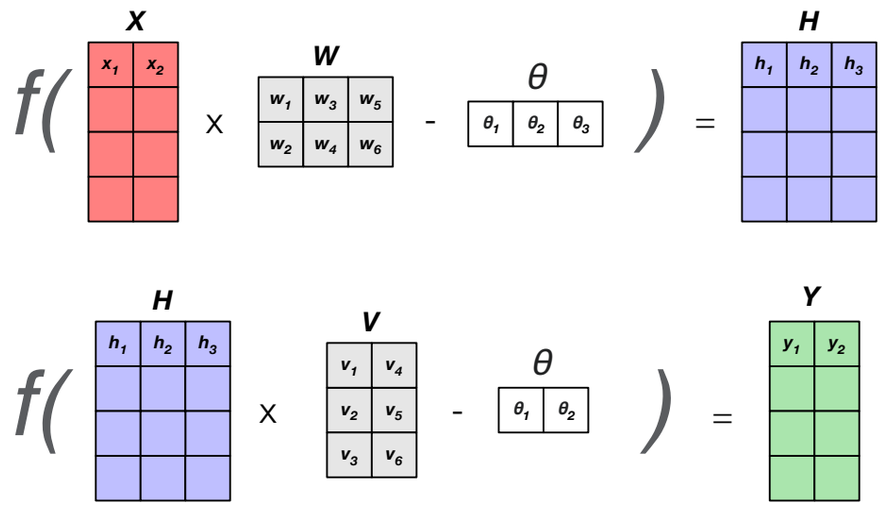

4
3
2
1
x_1
x_2
h_1
h_2
h_3
y_1
y_2
w_1
w_2
w_3
w_4
w_5
w_6
v_1
v_2
v_3
v_4
v_5
v_6



$$f\left(\begin{array}{|c|c|}\hline x_1 & x_2 \\ \hline & \\ \hline & \\ \hline & \\ \hline\end{array} \times \underset{W}{\begin{array}{|c|c|c|}\hline w_1 & w_3 & w_5 \\ \hline w_2 & w_4 & w_6 \\ \hline\end{array}} - \underset{\theta}{\begin{array}{|c|c|c|}\hline \theta_1 & \theta_2 & \theta_3 \\ \hline\end{array}}\right) = \begin{array}{|c|c|c|}\hline h_1 & h_2 & h_3 \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline\end{array}$$

$$f\left(\underset{H}{\begin{array}{|c|c|c|}\hline h_1 & h_2 & h_3 \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline\end{array}} \times \underset{V}{\begin{array}{|c|c|}\hline v_1 & v_4 \\ \hline v_2 & v_5 \\ \hline v_3 & v_6 \\ \hline\end{array}} - \underset{\theta}{\begin{array}{|c|c|}\hline \theta_1 & \theta_2 \\ \hline\end{array}}\right) = \underset{Y}{\begin{array}{|c|c|}\hline y_1 & y_2 \\ \hline & \\ \hline & \\ \hline & \\ \hline\end{array}}$$

```
X = np.array([[1, 2],
              [2, 1],
              [2, 4],
              [2, 3]])
W1 = np.array([[1, 2, 2],
               [2, 3, 1]])
t1 = np.array([1, 1, 2])
W2 = np.array([[4, 2, 2],
               [2, 1, 2]])
t2 = np.array([1, 2])
```



Solution

Pandas dataframe

```
import pandas as pd

df = pd.DataFrame()
```

```
df = pd.DataFrame().read_csv()

df = pd.DataFrame().from_dict()

df = pd.DataFrame().from_records()
```

Pandas dataframe

```
import seaborn as sns

df = sns.load_dataset('penguins')
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female
...	...	...	...	...	...	...	...
339	Gentoo	Biscoe	NaN	NaN	NaN	NaN	NaN
340	Gentoo	Biscoe	46.8	14.3	215.0	4850.0	Female
341	Gentoo	Biscoe	50.4	15.7	222.0	5750.0	Male
342	Gentoo	Biscoe	45.2	14.8	212.0	5200.0	Female
343	Gentoo	Biscoe	49.9	16.1	213.0	5400.0	Male

344 rows × 7 columns

Pandas dataframe

`df.columns` `pd.Index`

`df.index`
`pd.RangeIndex`

	species	bill_length	flipper_length	body_mass	sex
0	Adelie	39.1	181.0	3750.0	Male
1	Adelie	39.5	186.0	3800.0	Female
2	Gentoo	50.4	222.0	5750.0	Male
4	Gentoo	45.2	212.0	5200.0	Female

`df.values` `np.ndarray`

`dtypes` for each column

```
df.dtypes
```

```
species              str
island               str
bill_length_mm     float64
bill_depth_mm      float64
flipper_length_mm  float64
body_mass_g        float64
sex                  str
```

Access data

```
df.body_mass
df['body_mass']
```
→ pd.Series

	species	bill_length	flipper_length	body_mass	sex
0	Adelie	39.1	181.0	3750.0	Male
1	Adelie	39.5	186.0	3800.0	Female
2	Gentoo	50.4	222.0	5750.0	Male
4	Gentoo	45.2	212.0	5200.0	Female

```
df.loc[1]
```
→ pd.Series

```
df.loc[4, 'bill_length']
```
→ single value

Access data

```
df.loc[df.sex == 'Male']  → pd.DataFrame
```

	species	bill_length	flipper_length	body_mass	sex
0	Adelie	39.1	181.0	3750.0	Male
1	Adelie	39.5	186.0	3800.0	Female
2	Gentoo	50.4	222.0	5750.0	Male
4	Gentoo	45.2	212.0	5200.0	Female

```
df.loc[df.sex == 'Female', 'bill_length']
                                → pd.Series
```

Group

```
g = penguins.groupby('island')

g.mean(numeric_only=True)
```

island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
Biscoe	45.257485	15.874850	209.706587	4716.017964
Dream	44.167742	18.344355	193.072581	3712.903226
Torgersen	38.950980	18.429412	191.196078	3706.372549

Group

```
g = penguins.groupby(['island', 'sex'])
```

```
g.mean(numeric_only=True)
```

		bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
island	**sex**				
Biscoe	Female	43.307500	15.191250	205.687500	4319.375000
	Male	47.119277	16.597590	213.289157	5104.518072
Dream	Female	42.296721	17.601639	190.016393	3446.311475
	Male	46.116129	19.066129	196.306452	3987.096774
Torgersen	Female	37.554167	17.550000	188.291667	3395.833333
	Male	40.586957	19.391304	194.913043	4034.782609

Group

```
g = penguins.groupby('island')

g.describe()
```

	bill_length_mm								bill_depth_mm		...	flipper_length_mm		body_mass_g					
	count	mean	std	min	25%	50%	75%	max	count	mean	...	75%	max	count	mean	std	min	25%	50%	75
island																				
Biscoe	167.0	45.257485	4.772731	34.5	42.00	45.80	48.70	59.6	167.0	15.874850	...	220.0	231.0	167.0	4716.017964	782.855743	2850.0	4200.0	4775.0	53
Dream	124.0	44.167742	5.953527	32.1	39.15	44.65	49.85	58.0	124.0	18.344355	...	198.0	212.0	124.0	3712.903226	416.644112	2700.0	3400.0	3687.5	39
Torgersen	51.0	38.950980	3.025318	33.5	36.65	38.90	41.10	46.0	51.0	18.429412	...	195.0	210.0	51.0	3706.372549	445.107940	2900.0	3337.5	3700.0	40

Group

```
g.describe().T
```

	island	Biscoe	Dream	Torgersen
bill_length_mm	count	167.000000	124.000000	51.000000
	mean	45.257485	44.167742	38.950980
	std	4.772731	5.953527	3.025318
	min	34.500000	32.100000	33.500000
	25%	42.000000	39.150000	36.650000
	50%	45.800000	44.650000	38.900000
	75%	48.700000	49.850000	41.100000
	max	59.600000	58.000000	46.000000
bill_depth_mm	count	167.000000	124.000000	51.000000
	mean	15.874850	18.344355	18.429412
	std	1.820721	1.133116	1.339447
	min	13.100000	15.500000	15.900000
	25%	14.500000	17.500000	17.350000
	50%	15.500000	18.400000	18.400000
	75%	17.000000	19.000000	19.250000
	max	21.100000	21.200000	21.500000
flipper_length_mm	count	167.000000	124.000000	51.000000
	mean	209.706587	193.072581	191.196078
	std	14.142268	7.507366	6.232238
	min	172.000000	178.000000	176.000000
	25%	199.500000	187.750000	187.000000
	50%	214.000000	193.000000	191.000000
	75%	220.000000	198.000000	195.000000
	max	231.000000	212.000000	210.000000
body_mass_g	count	167.000000	124.000000	51.000000
	mean	4716.017964	3712.903226	3706.372549
	std	782.855743	416.644112	445.107940
	min	2850.000000	2700.000000	2900.000000
	25%	4200.000000	3400.000000	3337.500000
	50%	4775.000000	3687.500000	3700.000000
	75%	5325.000000	3956.250000	4000.000000
	max	6300.000000	4800.000000	4700.000000